B.Comp. Dissertation

Trusted Resource Metering for Outsourced Computation

By

Joyce Yeo Shuhui

Department of Computer Science

School of Computing

National University of Singapore

2020/21

B.Comp. Dissertation

Trusted Resource Metering for Outsourced Computation

By

Joyce Yeo Shuhui

Department of Computer Science

School of Computing

National University of Singapore

2020/21

Project No: H0041410
Advisor: Chang Ee Chien
Deliverables:
Report: 1 Volume

Abstract

This project studies the problem of verifiable outsourced computations. In particular, we are interested in a scenario where there is no mutual trust between the client (who wishes to execute the process) and the server (who helps execute the process in return for payment). We then propose a methodology for trusted metering, using a Trusted Execution Environment (TEE)-based Intel SGX solution. In our system design, the client first statically instruments the binary with information that can measure its resource use. Then, this instrumented binary is sent to the cloud provider to perform the computations. The server returns the computation output as well as a Trusted Log of what computations were executed. Using the Trusted Log, the client is able to verify that the server has indeed provided a minimum quality of service (QoS) and there is no over-charging. We examine the challenges relating to the immature technology of instrumenting SGX binaries, then give a proof-of-concept of our system applied to a Matrix Multiplication program. The result was that our system provides Trusted Metering, albeit at a high overhead. The overhead was mostly a result of the instrumentation, rather than the modelling or verification process.

Subject Descriptors:
D.4.6 Security and Protection

Keywords:
Trusted Metering, Trusted Execution Environment, Binary Instrumentation

Implementation Software and Hardware:
Intel SGX 2.9.1, Ubuntu Bionic 18.04.5 LTS, dynamorio 8.0.18719, dyninst 10.2.1, e9patch, Python 3.6

Acknowledgement

Professor Chang Ee Chein, my supervisor.

Gao Mingyuan and Hung Dang, who are part of Prof. Chang's research team.

Lee Wei Kang, who is concurrently doing his Final-Year project on Intel SGX-related work.

List of Figures

List of Tables

Table of Contents

Chapter 1

Introduction

A few years ago, Amazon Web Services (AWS) had a bug which erroneously overcharged many clients (CRN, 2019). While this issue was immediately rectified, it also brought to light the lack of transparency in cloud computing billing models. Cloud computing and outsourcing computation are becoming increasingly common. In such a model, clients pay compute nodes hosted on the cloud, in exchange for computational work to be done (Dang, Tien, & Chang, 2019). It is imperative to ensure that cloud clients are paying fair compensation for the computation service, and that they are not overcharged. Likewise, cloud servers should not be underpaid. In reality, this goal is not easy since there is no mutual trust between the client and cloud server.

There are two components involved in the billing process: (1) the measurement of the resource use ("metering") and (2) the pre-agreed payment policy. The policy is static, but the metering is prone to abuse. For instance, a naive measure for resource use would be to use the total running time (end time minus start time) of the process. With this approach, compute nodes can intentionally slow down the execution of the process by switching out of the process. The running time of the process is maliciously inflated, and the result is that the client is over-charged.

Additionally, we also have to consider which entity is responsible for the instrumentation process. Suppose that in the system above, the client is the entity responsible for inserting the code to log the start and end times. Then, the client can also abuse the trust it has by changing when the start and end times are taken, effectively underpaying the compute node.

Given, that there is fundamentally no trust between the client and compute node, then we need an unbiased source of trust. In particular, we are interested in solutions based on hardware-based trusted computing, also known as Trusted Execution Environments(TEEs). We will make use of the security properties guaranteed by the TEE to design a solution that can perform trusted metering of processes.

There are multiple metrics that can be measured from a process, ranging from running time, number of bytes sent through the network, number of bytes written and read from disk, and the number of instructions. After a series of preliminary investigations, we found a gap in binary instrumentation techniques which replace existing instructions in a compiled binary. The current techniques are either incompatible with trusted computing architecture, or have a large trusted code base (TCB) which makes them more vulnerable to attacks.

Furthermore, at the binary level, there are a large number of instructions being executed. One way to reduce the instrumentation overhead is to selectively measure the resource use of the process. In this project, we propose a sampled approach to achieve Trusted Metering. We use the measurement results from these sampled portions ("landmarks") to estimate the resource use for the entire process. We also make use of these measurements to provide a means for clients to verify the correctness and efficiency of the computation service provided by cloud servers.

1.1 Report Organisation

- Chapter 2 **Background**. We give an overview of the primitives and existing work relevant to designing a trusted metering system for the cloud.
- Chapter 3 **System Design**. Overview of our trusted metering system and how it enables clients to verify the quality of service provided by compute nodes.
- Chapter 4 **Binary Instrumentation with SGX**. Practical work done on instrumenting SGX binaries. We dive into why the existing techniques are not compatible with SGX.
- Chapter 5 **Trusted Metering**. Our implementation of the trusted metering system on a matrix multiplication program. We also show how our system is resistant to a compute

node slowing down the quality of service provided.

- Chapter 6 **Memory Optimisation**. We realised an implementation flaw and repeated the experiments from the previous chapter on our improved implementation.
- Chapter 7 **Multiple Landmark Types**. Considerations for extending to multiple landmarks, using a fast exponentiation program as an example.
- Chapter 8 **Performance Analysis**. Measuring the overhead of our system.

Chapter 2

Background

In this section, we review some background and related work on the topic.

2.1 Intel SGX

Intel(R) Software Guard Extension (Intel SGX) is a set of instruction code extensions on Intel CPUs, and is capable of hardware-based trusted computing.

Through Intel SGX, applications can run code in multiple isolated enclaves. Each enclave has a private address space that is only accessible by the enclave itself. This rule is enforced at the hardware level, and the processor will encrypt enclave memory. Programs then have two components, the untrusted application and trusted enclave. Untrusted applications can communicate with trusted enclaves via ECALLs, which are interface function calls part of the SGX design. Likewise, enclaves can communicate with applications via OCALLs. Intel SGX code can be compiled in two modes, simulation and hardware mode. Hardware mode requires the processor to be compatible with SGX and SGX to enabled in the BIOS.

When using the SGX SDK, the developer first generates the binaries for the application and enclave. This produces two binaries, `app.o` and `enclave.so`. The enclave binary is then signed to produce `enclave.signed.so`, and the application interacts with the signed enclave binary.

The signed enclave binary is trusted, and is guaranteed to run the code it claims to be running. This process can be verified using the SGX attestation mechanism. During attestation,

the enclave measurement hash is obtained by taking a 256-bit SHA2 hash of the enclave's internal state. The verifier can re-calculate this measurement and match the hash. When the hash produced is valid, the enclave has proven to a validator that it has been correctly instantiated. Any tampering with the enclave memory would cause the attestation process to fail. With attestation, a malicious cloud server cannot swap a short client process with a longer-running process and over-charge the client. Lastly, through the attestation process, a secure and authenticated channel between the validator and attesting enclave is established.

In short, Intel SGX provides isolated memory domains enforced by hardware. This property is useful for trusted metering because the (1) only the intended process is measured and (2) measurement results in memory cannot be altered by any process except the enclave itself. Furthermore, the metering result is valid because the metering code itself is run by the enclave and cannot be altered.

Unfortunately, being a hardware-based solution, the implementation of Intel SGX is vulnerable to side-channel attacks which can leak the private enclave cache and page table. These memory leaks can make it easier for an attacker to maliciously modify the metering results. However, we are not concerned with such attacks in this project. Intel has been actively working to mitigate against these attacks.

2.2 Instrumentation Techniques

We are interested in how we can measure or instrument the resource use of processes. The instrumentation process can occur at different levels, from source code to binary. We give an overview of each technique below.

2.2.1 Source code-based instrumentation

Instrumentation at the source code level involves inserting API calls to calculate the resource use. A compiler will parse the enclave source code, insert the instrumentation APIs, and produce an instrumented binary. This binary can still successfully attest itself because it was generated and signed based on the instrumented source code. At the source-code level, it is possible to

account for the following metrics: elapsed user time, elapsed system time, memory use, I/O use and network use of the process.

To account for the elapsed user and system time, two timers are started and stopped at the point of OCALLs and ECALLs. These functions signify a transfer of control between the application and enclave. The application running time is measured as the system time, and the enclave running time will constitute the user time.

The memory, I/O and network usage are measured in number of bytes. Since enclaves cannot directly perform these I/O operations, enclave code using these functions need to make OCALLs. The instrumentation library can then extract the number of bytes passed as arguments to the OCALL to account for these types of resource use. Such a system is known to achieve low performance overhead, low verification effort and a small trusted code base (Tople, Park, Kang, & Saxena, 2018).

However, there are limitations to trusted metering when instrumenting at the source code level. Most notably, the metrics used are more coarse-grained than binary-level instrumentation. With access to a compiled binary, it is possible to use more fine-grained metrics like the instruction count. Binaries can be instrumented statically or dynamically, and we examine both approaches below.

2.2.2 Dynamic Binary Instrumentation

When a client binary is instrumented dynamically, the number of instructions are counted as the code is being run. Dynamic binary instrumentation tools intercept and replace assembly instructions as they are being executed. This process is known as dynamic binary translation (DBT).

In DBT, the instrumenting program needs access to the enclave memory in order to add trampolines and replace instructions as the process is being run. Since the enclave memory is only internally accessible by itself, the instrumenting program and client process must then executed within the same enclave environment.

Even when within the same enclave, the instrumenting program is altering the code that the

client process runs. This behaviour is not compatible with the design of SGX enclaves, which (in principle) cannot be modified dynamically. There are a few issues to address before this task can be done. For instance, SGX enforces static memory partitioning so the size and permissions of enclave memory cannot be changed at run-time. This limitation requires the instrumenting program to statically generate all the required memory pages to run the client process.

One approach to tackle these challenges is to first create a separate system addressing the compatibility issues of DBT in SGX. Examples of such systems include Graphene SGX (Tsai, Porter, & Vij, 2017) and Ratel (Shinde, Cui, Sen, Yuan, & Saxena, 2020). By creating workarounds for the SGX restrictions, these systems extend the types of processes that can be run in enclave environments. In particular, the Ratel system modifies DynamoRIO to enable DBT in SGX enclaves. The instrumenting program and client process can then be run within Ratel to meter the code. The result was that the Ratel system had a high coverage and portability for performing DBT on enclave binaries. Unfortunately, the design of Ratel sacrificed performance, making the system too inefficient to be used for outsourced cloud computations.

On top of that, there are two key security issues with using Ratel. (1) Initially, the client's trusted code base (TCB) only comprises their submitted program. The instrumentation process adds in both the instrumentation program and compatibility program to the TCB. This large TCB counters the original intention to run the instrumentation process within an enclave environment. (2) Besides creating workarounds for SGX memory restrictions, the Ratel design also involves modifying the SGX SDK. While there are no immediate attacks arising due to these workarounds, discarding the SGX design and security properties only exposes a greater attack surface.

These problems suggest that dynamic binary instrumentation in SGX is not feasible. As a result, we look to static binary instrumentation instead.

2.2.3 Static Binary Instrumentation

Static binary instrumentation involves inserting instrumenting code to an enclave binary, producing a new enclave binary. The new enclave binary is then signed to produce a signed enclave

binary. There are numerous static binary rewriting tools, but not all are compatible with SGX. We use two such tools in this project, dyninst (Ince & Hollingsworth, 2013) and e9patch (Duck, Gao, & Roychoudhury, 2020).

Dyninst is widely used, and offers more functionality than e9patch. Most notably, dyninst can process the binary in basic blocks, which are sections of assembly code with only one entry and one exit point. With dyninst, it possible to introduce instrumentation functions at the start and end of basic blocks.

On the other hand, e9patch rewrites binaries using lower-level binary rewriting techniques which do not change the addresses of existing instructions. This technique greatly improves compatibility with cross-binary code such as the case of using SGX. The application can continue to interact with an instrumented enclave binary without having to rewrite the address references to the enclave binary.

Unfortunately, e9patch is limited because it only parses information at the instruction level. e9patch scans the binary for the instruction specified, then instruments the instructions found (example: replace all jump with a print). It is not possible to directly instrument blocks of instructions at once, or instrument based on the basic block structure. Despite these limitations, the approach taken by e9patch to rewrite binaries without moving jump targets is most viable for our goal of enclave instrumentation.

Fundamentally, source code-based instrumentation and dynamic binary instrumentation have conceptual limitations in their approaches. While current methods in achieving static binary instrumentation have not been very successful, this technique would be most feasible moving forward.

2.3 Sampled Instrumentation

With these preliminaries in mind, we can design our static binary instrumenting system for trusted metering. At the binary level, we want to start by counting the total number of instructions. However, counting the total number of instructions one at a time and instrumenting the entire process can be inefficient. One optimisation is to count at the basic block level. Stati-

cally, the number of instructions within a basic block is counted. When the process executes and reaches a basic block, only the aggregated number instructions within the basic block needs to be added to the final tally. However, such a task is currently not possible with our limited tools. e9patch, which is more compatible with SGX than dyninst, is unable to recognise basic blocks. With e9patch, we can only count instructions one at a time.

Counting every instruction in a process is extremely inefficient. To reduce the overhead of instrumentation, we are interested in exploring a sampled approach to instrumentation. With sampling, only selected sections of the process will be instrumented. We then measure the resource use of the sampled sections, and use the measurement to estimate the resource use of the entire process.

Sampling can be done against time or against the basic blocks. Time-based sampling involves instrumenting the process based on the time of execution. For example, when the 30th second is reached, sampling begins. Sampling then stops at the 70th second. To implement such a system, we would need two versions of the client program, one instrumented and one uninstrumented. Depending on the time of execution, we can make the context switch to the correct process. During these context switches, the memory of ongoing computations needs to be copied between the two processes. The copying process is not trivial, especially if the two processes have differing memory layouts due to the additional instrumentation code.

Sampling is not new in the domain of trusted metering. OVERSEE is a system that verifies the correctness of compute node execution using a sampling-challenging mechanism (Cai, Shi, Yuan, Liu, Zheng, Chen, Li, Leng, & Guo, 2020). Before a program is sent to the compute node, sampled sections of the program are executed and timed. The compute node then executes the entire program, which will also output the running time of the sampled sections. A benign compute node will produce an execution time that does not differ significantly from the pre-measured execution time of each section. Since the sections that are being sampled are not known before hand, it is difficult to cheat the system. The compute node would have to determine when sampling is done and not done, and selectively inflate resource use only when the process is not sampled.

However, the OVERSEE method requires the sampled sections to be executed beforehand. It is not always feasible to do so, especially if the sampled sections rely on the unsampled sections to first be executed. It is even more unrealistic to also execute the unsampled sections beforehand, since that defeats the purpose of outsourcing the computation in the first place. Furthermore, OVERSEE's only metric for resource use is to measure execution time, which is the easiest to attack just by way of a malicious operating system's scheduling policy.

Nevertheless, we can extend the sampled instrumentation techniques in OVERSEE to incorporate both execution time and instruction count metrics. The purpose of incorporating execution time is to be able to extrapolate the metering results to the entire process.

Chapter 3

System Design

With these preliminaries, we can design a system that keeps the SGX requirements and trusted metering requirements in mind.

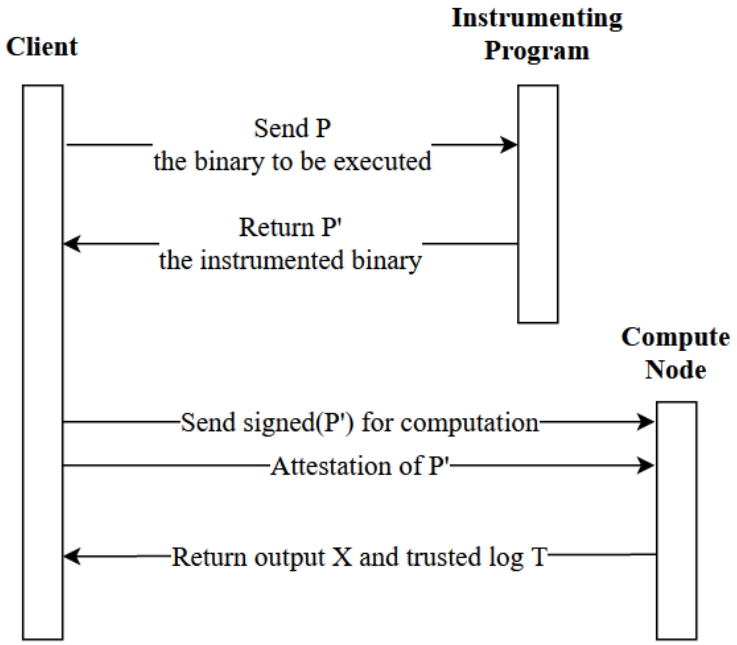


Figure 3.1: System Design

In the deployment of static binary instrumentation, the client has an uninstrumented program P. The binary P is submitted to a third-party instrumenting program, which produces an instrumented binary P'. The instrumenting program is public, and incentivised to be benign to maintain public trust. As such, the compute node also trusts the instrumented program.

The instrumented program P' is then submitted to the compute node for execution. The client is able to verify that the compute node is indeed running P' via the remote attestation process of Intel SGX. The attestation process also results in a secure and authenticated channel between the client and the compute node. This channel is then used to send the computation output and instrumentation output back to the client. Based on the instrumentation output and the pre-agreed payment policy, a final billing amount is generated for the client.

3.1 Instrumentation Design

In our system, we only want to selectively measure the resource use of the code. We require sampled sections to be within a basic block so that **when a sampled section runs, the entirety of the sampled section is guaranteed to be executed**. We call these sampled sections **landmarks**. Each landmark is associated with a start and end point for sampling, and these indicators are inserted during the instrumentation process. Currently, we do not have a generalisation strategy for landmark selection. We manually select landmarks based on known properties of the program, and show the considerations made in doing this landmark selection.

After the instrumenting program inserts the landmark indicators, we have to measure the resource use of the landmark. We use two metrics:

1. The total number of instructions within a landmark, which can be determined statically. A parser will read the enclave binary and record the length of various landmarks. This value is used to compare the relative size of various landmarks.

2. The start and end time of the landmarks. We do not want to incur extra overhead by running the sampled sections beforehand. As such, we only measure the landmark duration of the program instances $signed(P')$ as they are executed by the compute node. During the process execution, a trusted log is produced and a log entry is added each time a landmark is visited. Note that if a landmark is visited twice during one execution of a process, then there will be two sets of timestamps being generated.

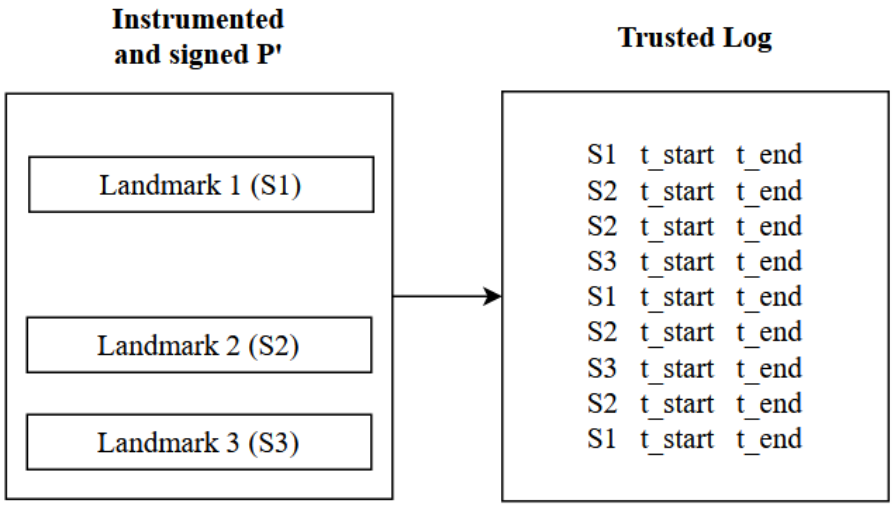


Figure 3.2: Example of Trusted Log generated as $signed(P')$ executes

As shown in Figure 3.2, the landmarks are not guaranteed to execute in any particular sequence and each landmark can be executed more than once, depending on the program control flow.

3.2 Compute Node Billing

The result of the process execution is a trusted log that meters the resource use of the instrumented landmarks. The next step is to extrapolate the trusted metering results to the entire process. For this task, we need to know either the distribution of landmarks within the process or some metric about the process itself (and not its landmarks). We opt to use the second option, and use the coarse-grained measure of the total process execution time.

Based on the total execution time of the process and the execution time and instruction counts of the landmarks, it is possible to estimate the amount for billing just by linear regression on two variables: (1) total number of landmarks and (2) total running time of landmarks.

For each type of process, we create a generalised model by pre-running some samples on a trusted node. This trusted compute node's resource use is the reference point for billing. The trusted node's work only needs to be done once. Afterwards, any instance of the program (on any input) can be billed without the trusted node. Here, we argue that the need for a static

root of trust is necessary, because there has to be some reference point for fair billing.

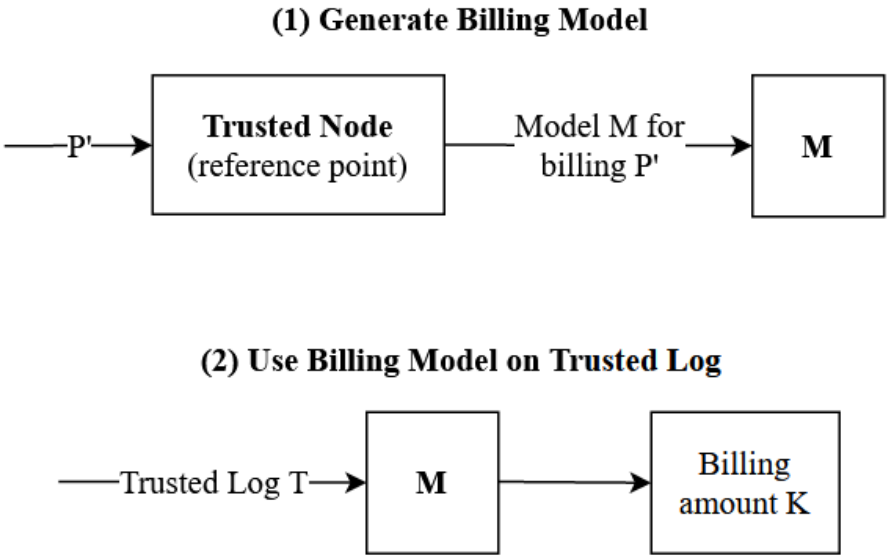


Figure 3.3: Fair Billing Model

For the model to be fair, there has to be a sufficient number of landmarks spread throughout the process. For instance, we cannot just use one landmark at the start of the program that only executes once regardless of the input size. We show how such landmark selection can be done for a single-landmark program in Chapter 5, and for a multiple-landmark program in Chapter 7. In our system design, a trusted third party is responsible for selecting landmarks. This design, compared to the client including the landmark, ensures that the client does not maliciously omit landmarks for billing.

Furthermore, to get a sufficiently accurate model for estimation, many instances of program P with multiple inputs need to be run beforehand. This methodology can create some overhead, which we evaluate in Chapter 8.

3.2.1 Partial Billing

One source of concern is the need for partial billing. In other designs of trusted metering (Cai et al., 2020), the server is never rewarded for partial computation. These designs all place full trust in the client. However, we also want to account for the case of a malicious client who could intentionally make the process crash after getting the desired output from partial computations.

Our system enables partial billing of clients. Since the billing model uses linear regression, then any partial logs still produces proportionately valid billing amounts. Hence, the compute node can still return the client partial logs on crash and request for billing. In the billing policy, the client may opt to penalise the server for partial computations.

3.2.2 Logs without Execution

Lastly, a key problem is that the compute node can produce logs without ever performing the computation service. Then, the client would be charged for no work being done. We can counter this malicious server behaviour by requiring the logs be signed during enclave execution. Within entering the enclave and executing the enclave code, the compute node would not have access to the signing key. We use the same secret key that is used to verify program correctness (see next section below).

3.3 Client Verification of Process Output Correctness

When an enclave executes, its control flow cannot be externally modified. Hence, when a compute node runs program P', it is guaranteed to run it in the way that the client had intended it to. However, enclave execution does not guarantee that the process will terminate at the intended exit point.

One way to ensure that the process reaches the last statement in the enclave is for the client to include a unique output like printing a statement to indicate that the process has completed. However, the compute node can also maliciously insert the same secret output at the end of the log, without ever running the entire program at all. This verification relies on the compute node not knowing what the secret output the client has used, and it is achieving Security by Obscurity. After sufficient instances, the compute node can guess the secret output, making this method not ideal.

Instead, we want to use the guarantees of enclave execution to check if the process terminates as intended. The compute node can sign the process output and logs using a private key generated within the enclave (Cai et al., 2020).

At the start of enclave execution, a public-private key pair is generated. The private key is only used at the end of enclave execution to sign the output X, producing a signed output X'. The public key is placed in the reserved field of the SGX report. During attestation of the process P', the client can retrieve the public key placed in the SGX report. This public key can then be used to verify the signed process output X'..

The compute node cannot simply generate its own public-private key pair and sign the output because the public key needs to be in the SGX report. The generation of the keys and placing of the public key in the SGX report is only possible during enclave execution. During the attestation process, the client is able to verify that the compute node is performing the key generation task as specified.

Lastly, we do not want the responsibility of process correctness to be on the client. Hence, this methodology described here is also achieved by the instrumenting program in Figure 3.1.

3.4 Client Verification of Minimum Quality of Service (QoS)

When a client sends program P' to the compute node for execution, a dispute will not occur if the reference node already has a billing amount for that particular program with that particular input. However, suppose that the reference node has no such value, then the client can dispute the billing amount.

The client needs a means to compare their trusted logs with the sampled trusted logs. Clearly, there will not be any trusted log with the identical landmark order available, else the client could have just gotten the billing amount. However, there will still be identical sections of landmark patterns that can be compared. Figure 3.4 shows how a landmark pattern of size 4 can be compared across two trusted logs on two different inputs.

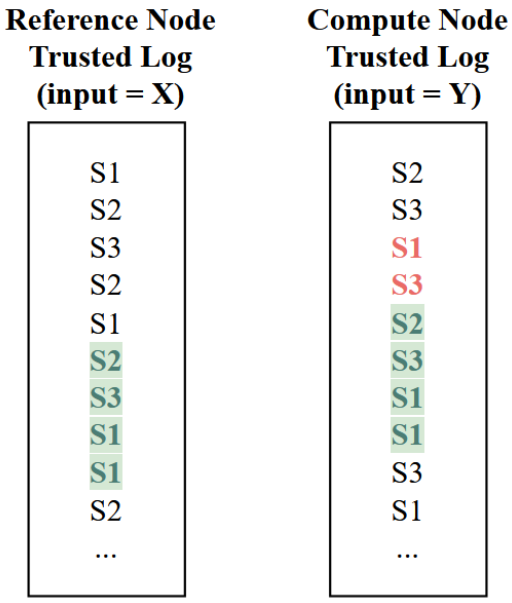


Figure 3.4: Matching of Landmark Patterns from two different logs

Ideally, longer landmark patterns make for meaningful comparisons. However, we can break the process logs up into multiple landmark patterns for matching, so there are more opportunities for matching. Such a design only works for iterative programs where landmarks can be repeated a sufficiently large number of times. This assumption is viable in the case of cloud computing, where computations are often repeating the same computation patterns.

The next problem questions if such landmark overlaps definitely exist, and how long the log must be to guarantee the existence of all patterns. We first define the following variables:

- k, the number of landmark types.
- n, the minimum size of a landmark pattern for matching.
- L, the number of log entries in the reference node.

Firstly, the smallest possible log size L that contains all landmark patterns of size n would be a variant of a de Brujin sequence. $L_{min} = k^n + (n-1)$, where we add $n-1$ to account for the non-cyclic nature of our logs. The first solution is to then carefully curate an input such that its log would be a de Brujin sequence, the most effective mininmum reference log. However, this task is not feasible given the large number of logs we expect to see and the complexity of running a program.

Our alternative is to then run on as many random inputs as possible. Given the compute node's trusted log, we try as much as possible to match as many landmark patterns as possible. Each landmark should be part of at least two different sequences of length two, to account for the non-landmarked time before and after the landmark. In Figure 3.5, Pattern 1 accounts for the green non-landmarked section and Pattern 2 accounts for the red one.

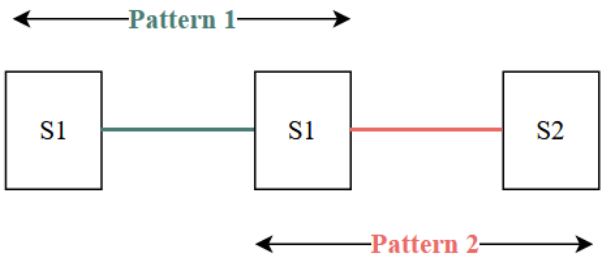


Figure 3.5: Each landmark has to be part of two landmark patterns

However, such a method of running against random inputs gives no guarantee of containing all permutations of landmark patterns. Suppose that there are landmarks not part of any landmark sequence, then the non-landmarked portions cannot be accounted for. In Figure 3.4, there are two unaccounted areas between the landmarks $S1$ and $S3$. These time periods are when the server could arbitrarily slow down, for even an infinite amount of time (by pausing the process and running others in the background). If the client suspects such large-scale fraud, they may request the reference node to perform the computations on their input. In the long term, the reference node would eventually collect all such landmark patterns.

In the next few sections, we propose a variety of metrics that our system is able to collect from the landmarks. Depending on the billing policy, the client may use a subset or all of these metrics to verify the compute node's work.

3.4.1 Verification of Landmarks

We want to ensure that the billing is fair. During the generation of the billing model M for program P', each landmark in P' is executed multiple times. Hence, the execution times of landmarks are known. The compute node cannot act maliciously during landmarks, since the trusted log would indicate that the landmark has an inflated running time.

Given the execution of P' by the compute node, let $T_{start_1}, T_{start_2}...$ be the trusted starting times of the various landmarks. The timings are trusted because they were generated in the enclave, but they are not guaranteed to be the most efficient timings achievable by the compute node. Now, we use the trusted landmark execution time given by the reference node which generated the billing model. Then, we have another set of trusted timestamps $t_{start_1}, t_{start_2}....$

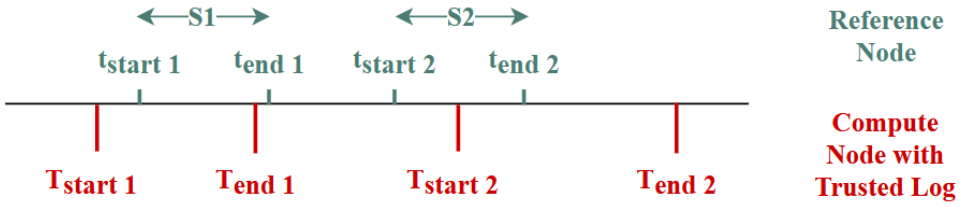


Figure 3.6: Visualisation of Timestamps at Verification Time

Given a trusted log as in Figure 3.2, we can calculate the average landmark duration for each set of landmarks $S1$, $S2$ and so on. Then, we can calculate the inflation, or percentage difference between the average landmark times in the log and from the reference node.

The inflation I of period i is defined as:

$$I_i = \begin{cases} \frac{T_i - t_i}{t_i}, & \text{if } T_i > t_i \\ undefined, & \text{otherwise} \end{cases}$$

3.4.2 Verification of Non-Landmarked Sections

Theoretically, the landmarks are randomly placed so the adversary cannot predict when the process is landmarked and when it is not. This design makes it hard for the compute node to selectively be malicious during non-landmarked times. Nevertheless, we also want to ensure that the sections of code which are not landmarks can be reliably executed with a minimum QoS. For this task, the client can use the timestamped values of the landmarks.

There are two metrics obtainable from the timestamps, and both are jointly used as a guideline to ensure fair billing.

Net deviation

The deviation d_i of each landmark i is defined as:

$$d_i = T_{start_i} - t_{start_i}$$

The net deviation d is simply the summation of all the deviations (positive deviations in red and negative deviations in green).

$$d = \Sigma_i d_i$$

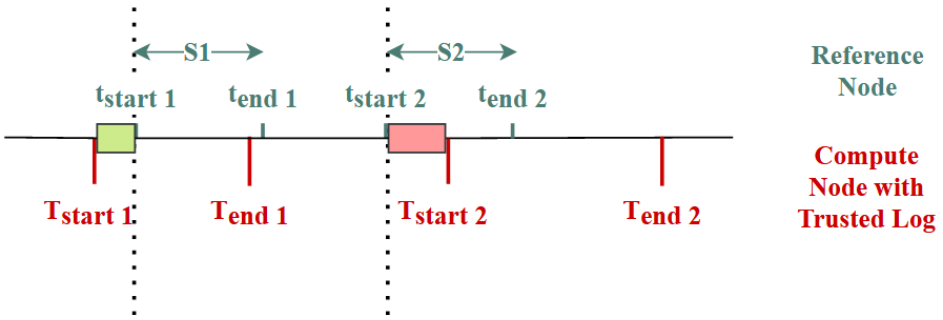


Figure 3.7: Visualisation of Net Deviation

The net deviation is designed to account for overall service quality. Deviation at each landmark i can be both positive (red) or negative (green). A positive deviation indicates that the compute node was slower than reference execution in the antecedent unmarked section. A negative deviation indicates that the compute node was faster than reference. Using net deviation as a metric can help compute nodes be more flexible in assigning resources to various processes. Suppose a process is slowed down in one section, the compute node can make up for it by running quicker in a later section.

However, net deviation is prone to the avalanche effect, where a slowdown in an early landmark is repeatedly taken into account. In Figure 3.8, the compute node was only slow for the period before landmark $S1$. The deviation is over-accounted for in the succeeding landmarks.

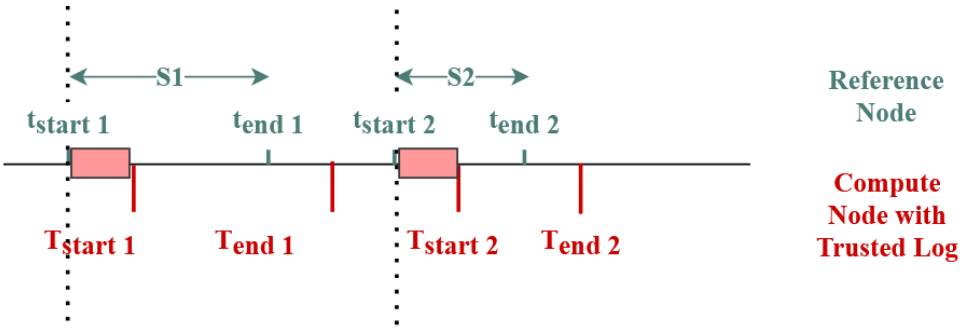


Figure 3.8: Avalanche Effect of Net Deviation

The remedy for this avalanche effect is to align the end times of the preceding landmark. The adjusted deviation d_{adj_i} of each landmark i is defined as:

$$d_{adj_i} = d_i - (T_{end_{i-1}} - t_{end_{i-1}})$$
$$= (T_{start_i} - T_{end_{i-1}}) - (t_{start_i} - t_{end_{i-1}})$$

In effect, the adjusted deviation is comparing the duration of the non-landmarked section in both nodes. While more accurate, the adjusted deviation is more time consuming to calculate than just the deviation, involving about twice the number of computations. We can also adopt the method of how landmarked sections are compared, and find the inflation rates for non-landmarked periods.

3.4.3 Verifying Consistent QoS

However, in certain applications, the client requires that the compute node consistently provides a minimum QoS, and not just achieve a net result. Then, the client has to verify that the compute node's timestamps are consistently hitting the reference timestamps. This can be achieved by considering only landmarks when the compute node took longer for a section, whether it is landmarked or not landmarked.

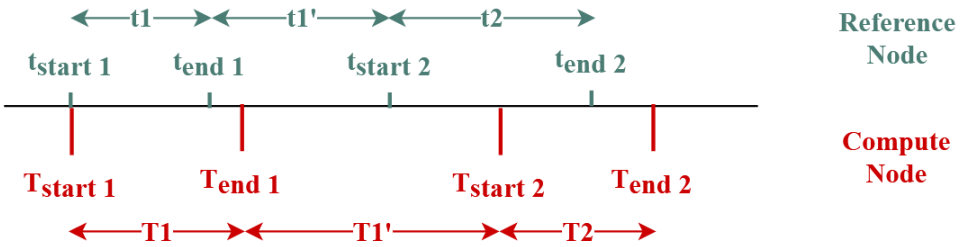


Figure 3.9: Verifying Consistent QoS Example

Reference Node (green)	**Compute Node** (red)	**Inflation** (%)
t1 = 3s	T1 = 4s	33.3
t1' = 4s	T1' = 6s	50
t2 = 5s	T2 = 4s	-

Table 3.1: Logged section duration for Example in Figure 3.9

Recall that the goal is to ensure a consistent QoS is being offered by the compute node. Then, we can count the proportion of running time where there is an inflation above a set margin. However, it would be more interesting to account for the magnitude of these inflation values. For the measure of consistency, we want to penalise large inflation rates more than small inflation rates. We can do this by taking the inverse of the harmonic mean as a metric. The table below illustrates why this measure is best compared to other mean metrics like the arithmetic mean (AM), geometric mean (GM) and harmonic mean (HM).

Inflation Rates (%)	**AM**	**GM**	**HM**	**Inverse of HM * 100**
50, 50, 50	50	50	50	2.0
20, 50, 80	50	43	36	2.8
5, 5, 140	50	15	7	13.6

Table 3.2: Inverse of HM heavily penalises large inflation rates

We multiply the metric by 100 to make comparing the values easier. The client can also opt for alternative mean metrics presented in the table to penalise other forms of behaviour.

Chapter 4

Binary Instrumentation with SGX

With the theoretical system design set aside, we detail the work done and findings on binary instrumentation specific to SGX. Binary instrumentation is a technique that is widely-researched on. However, many of these techniques are not compatible with SGX. We reveal key issues surrounding the design of SGX-compliant systems, which will inform how a trusted metering system should be design.

4.1 SGX Compatibility of Existing Tools

4.1.1 Dynamic Binary Instrumentation (DBT) with DynamoRIO

In the background section, we mentioned the issues surrounding the theoretical use of DBT in enclave environments. We expected the DBT engine to fail when used with SGX because of the inability to access enclave memory.

We verify and investigate the issue using DynamoRIO (Bruening, Zhao, & Amarasinghe, 2012), a powerful and extensive tool for DBT. We applied DynamoRIO's instruction count plugin to `SampleEnclave`, which is included in the SGX SDK. The result was that the DynamoRIO engine goes into an infinite loop. With premature termination, the process output is that billions of instructions have been executed.

```
Client inscount is running
^CInstrumentation results: 18681315924
```

The diagram below shows the stack of processes that were running. The DBT engine is loading and running an application, which in turn loads and runs an enclave.

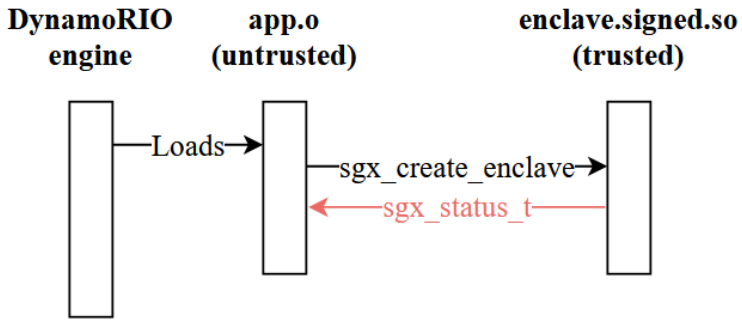


Figure 4.1: DynamoRIO execution of `SampleEnclave`

We did not have access to a debugger that could (1) debug a DBT engine (2) when the DBT engine is running `app.o` (3) which was loading the enclave `enclave.signed.so`. With current tools, it is not possible to directly debug the behaviour of the application or the enclave. Nevertheless, our investigation revealed that the enclave was never even initialised under the DBT engine.

In a regular execution without the DBT engine, a call to `sgx_create_enclave` always returns a `sgx_status_t` status code. When the status code is not `SGX_SUCCESS`, the application terminates. Under the DBT engine, the enclave initialisation failed but no status code was returned.

This suggests that the disassembler in the DBT engine was not able to parse SGX instructions. Correspondingly, when running the application, DynamoRIO replaces the SGX extension instructions in `app.o` with a jump instruction whose operand is its own address. The DBT engine never receives `SIGTERM` from the application, resulting in the infinite loop.

Unfortunately, there is currently no well-built DBT engine that can support SGX instructions. With support for SGX architecture, more work can be done to better understand how DBT can be achieved in SGX without the use of inefficient compatibility systems like Ratel (Shinde et al., 2020). Hence, our trusted metering system cannot be designed around dynamic binary instrumentation.

4.1.2 Static Binary Instrumentation with Dyninst

Next, we investigated the static instrumentation tool, Dyninst (Ince & Hollingsworth, 2013). Static instrumentation of the unsigned enclave produces a new enclave file, `enclave.instr.so`. This new file then needs to be signed by the user again, before it can be used by the application.

Figure 4.2: Overview of Static SGX Binary Instrumentation

Initially, we wanted to do the instrumentation with Dyninst as it already has basic block parsing features. This would allow us to create higher-level systems around the basic block abstraction. Unfortunately, we found compatibility issues with Dyninst and SGX.

The Dyninst tool was able to parse and instrument the enclave binary without error. However, the output binary was not executable as it resulted in `sgx_status_t SGX_ERROR_UNEXPECTED` being returned. Since this output binary is an enclave file, its memory regions are protected and hence it cannot be debugged with `gdb`. We have to assign the debugger access to the SGX hardware using `sgx-gdb`, so that it can read the enclave memory. As a protective feature, the enclave binary also needs to be compiled with the `DisableDebug` feature set to `0`. From debugging, we found that the binary did not have the correct ELF format.

```
readelf: Warning: [ 4]: Unexpected value (1) in info field.
readelf: Warning: [ 7]: Unexpected value (2) in info field.
```

The section `[ 4]` in the section table is `.oynsym`. Dyninst moved the original `[ 4] .dynsym` section to `[ 30]`, and replaced it with a new section `.oynsym`. The original `.dynsym` contained global symbols not in `.syntab`.

The new section `.oynsym` has a `sh_entsize` value of `0x18`. A non-zero value in `sh_entsize` implies that the section is a table of fixed-size entries, with each entry occupying `0x18` bytes. The entity size here is exactly the same as in the original `.dynsym`, suggesting that `.oynsym` is also serving as a symbol table. Knowing this information helps us interpret the other section header table fields.

We get the following information about `.oynsym` in the instrumented file:

```
Address
Flags  Link  Info
A       5       1
```

The `sh_flags` has value `A`, which corresponds to `SHF_ALLOC`. The `sh_link` field corresponds to section header index `5` of the associated string table. Then, the `sh_info` field, the field causing the readelf error, needs to be an increasing sequence of numbers representing the symbol table index.

Initially, the `.dynamic` section contained the zero-entry for section header index `5`. However, dyninst changed the section header index for `.dynamic` to `30`, creating a missing zero-entry. As a result, the info field for both sections `[ 4]` and `[ 7]`, which had `sh_info` indices `1` and `2`, were incorrect. While we can go on to change the linker in dyninst, it would deviate from the main goal of the project, which is to design a trusted metering system. Nevertheless, our investigation has identified the problem with Dyninst's SGX incompatibility.

4.1.3 Static Binary Instrumentation with e9patch

Lastly, we investigate e9patch, which uses lower-level binary rewriting tools. This technique means that e9patch does not need to rewrite the section table like Dyninst, which could avoid the incorrect linking information faced when using Dyninst. We test the e9patch tool to patch all instructions in the `SampleEnclave` with instruction printing and the output that most instructions were successfully patched.

```
num_patched             = 29200 / 29341 (99.52%)
```

We verify that the section header table index of `.dynamic` was still `5`. When executed after resigning, the enclave was also able to successfully be created (`sgx_create_enclave` and destroyed (`sgx_destroy_enclave`). However, no instructions were printed since the enclave environment has no access to untrusted I/O syscalls. We further investigate I/O instrumentation for the trusted log in Section 4.3.

However, e9patch is not fully compliant with all SGX features. In hardware mode, we were successful on `SampleEnclave`, but not enclaves with attestation and sealing features,

namely `LocalAttestation` and `SealUnseal` in the SGX SDK sample enclaves. The underlying issue is that e9patch uses the Capstone disassembler, which was not able to disassemble the binaries `libenclave_initiator.signed.so`, `libenclave_responder.signed.so`, `libenclave_seal.signed.so`, `libenclave_unseal.signed.so`.

```
warning: failed to disassemble (.byte 0x06) at address 0x14f29
error  : failed to disassemble the .text section of
    "/home/joyce/LocalAttestation/bin/libenclave_initiator.signed.so";
    this may be caused by (1) data in the .text section, or (2) a bug in the
    third party disassembler (capstone)
```

In all of there binaries it was the byte `0x06` which could not be disassembled, albeit at different addresses (`0x18909`, `0xba29` and `0xb9a9` respectively. However, at all of those addresses, we would find the byte sequence:

```
00014f20: 0f0e 0d0c 0b0a 0908 0706 0504 0302 0100
```

The successfully-instrumented `SampleEnclave` had no such byte sequence. This could be a placeholder sequence used by e9patch for unknown instructions. Next, when the enclaves were compiled in simulation mode of SGX, e9patch was not able to disassemble the `SampleEnclave` binary. The byte that could be disassembled was different from the enclaves in hardware mode.

```
warning: failed to disassemble (.byte 0x2e) at address 0x22102
```

4.2 Trusted Count

From the system design, we extracted two features of landmarks which we want to consider, the instruction count and timestamps. The first instrumentation task is to perform instruction counting within the landmark, which we term Trusted Count. This task can be done manually, although it scales with the number of landmarks. We disassemble the whole program in sgx-gdb and look for the landmark indicators of each landmark. We inserted the landmark indicators at the source-code level to view the debug symbols.

```
0x0000000000001150 <+29>:    callq  0x1125 <start_instrument_S1>
0x0000000000001155 <+34>:    mov    -0x4(%rbp),%eax
```

```
0x0000000000001158 <+37>:    add    %eax,-0x8(%rbp)
0x000000000000115b <+40>:    mov    $0x0,%eax
0x0000000000001160 <+45>:    callq  0x112c <end_instrument_S1>
```

Then, we can record that landmark $S1$ has three instructions. The instruction count of each landmark will be used to weigh the different landmarks in the billing model.

The alternative to this manual parsing process is to use an in-memory instruction counter that counts the number of instructions as the process is executing. The fastest way to perform the addition would be to use a counter. As there is a large number of instructions in a binary, the count would also be very large. We need to use a floating point Streaming SIMD Extensions (SSE) register, which was not accessible to us. As such, we used the static parsing method instead.

4.3 Trusted Log

The main concern about designing an SGX-compliant trusted log method is the I/O. Note that we are doing this task with instruction-level instrumentation. The instrumentation process is as follows: (1) log to an in-memory location each time an instruction is executed and (2) pipe the final log to I/O on executing the last landmark. This design means the last instruction has to be instrumented differently to add the I/O functionality. However, this is not possible when we are doing instruction-level instrumentation.

Note that our design involves each landmark being executed multiple times. With reference to Figure 3.2, $S1$ is the last landmark being executed. Statically instrumenting $S1$ with I/O functionality would mean that the first and fifth entries of the log would also be piped to I/O. The design is more problematic when there is only one landmark being executed multiple times, since each landmark execution would trigger I/O. Nevertheless, our current technique is to instrument every landmark with I/O functionality. Later on, we can see how this I/O functionality can be optimised.

We demonstrate this task by attempting to print every instruction executed by a binary. Outside the enclave environment, we can verify that such a task is trivial. In the enclave

environment, we have access to a tool like `e9patch` that can already replace instructions within SGX without error. The instrumented program can still run to completion, but there is no actual I/O. This discrepancy is because the enclave environment has no access to I/O syscalls, other than via stipulated OCALLs.

First, we replace the syscall with an OCALL. We defined and compile and OCALL in the same binary as the enclave, so that the enclave has access to the compiled OCALL. We instrument the instructions to be OCALLs, but the following error is thrown:

```
error  : failed to embed ELF file "enclave"; file uses relocations
```

Relocation is a process in binaries to help link multiple symbolic information saved from the source code. The enclave is a shared object file with `.so` extension, so it is relocatable. The current technique for instrumentation involves moving the existing instructions to a separate resource section, then setting a binary trampoline to that resource section. Pointing to a resource within the same binary would mean that the shifting process has to take into account not to move this instruction, or to move it and find its new address while it is loaded in memory.

One reason why the enclave would have relocation information is because it is referencing the proxy functions for the OCALLs to the application. Recall that it is possible to instrument the enclave with a function from a regular binary. Only when instrumenting with an OCALL did the relocation problem arise. As such, we change the reference to an internal function within the enclave that is not an OCALL. However, even doing so, we get the relocation problem. This implies that we cannot do instruction-level instrumentation to point back within the same file. We test this technique entirely outside the enclave environment and verified that it is true: we cannot instrument a program to point back to itself.

The next logical step is to then attempt compiling the OCALL outside of the enclave binary. We would then have to link `enclave.so` with the OCALL that has I/O functionality, to produce a new enclave file. However, we would also have to do an additional step of rewriting the OCALL table, which defines the exiting interface functions an enclave is allowed to call. We were unfortunately not able to perform this task as the SGX SDK wraps the OCALL table functionality inside the Edger8r tool.

Chapter 5

Trusted Metering

In this section, we implement the trusted log system for a matrix multiplication program. As this work was done concurrently with studying binaries, we used source-code instrumentation to implement this part. In particular, we show how landmark selection can be done, and how the trusted logs generated from the landmark can be used by the client to verify the billing requested by the compute node.

5.1 MatrixMultiplication program

We use an $O(n^3)$ iterative sequential algorithm for matrix multiplication, and only insert one type of landmark. The landmark location is shown in the python pseudocode below, which multiplies two matrices `m1` and `m2`.

```
for i in range(num_rows(m1)):
    for j in range(num_cols(m2)):
        for k in range(num_cols(m1)):
            start_instrument()
            m3[i][j] += m1[i][k] * m2[k][j]
            end_instrument()
```

For simplicity, we assume that the number of rows and columns in both matrices are the same. Theoretically, the number of landmarks executed is directly proportional to the running time. However, the instrumentation overhead will be very high. As such, we only instrument

a portion of the landmarks, every 1 in 20, reducing the instrumentation amount by 95%. Depending on the desired overhead, the client may adjust this value. Having more landmarks will increase the probability of detecting inflated billing but cause a higher overhead.

We also considered if the 5% of landmarks should be randomly selected, so the compute node cannot predicted the instrumentation periods. However, random landmark placement will skew the inter-landmark time. The landmark fingerprint $S1 - S1$ can have very different running times for the non-landmark portion, so the client cannot verify the running times of non-landmark sections.

Lastly, the instrumentation of one line of source code produces a time recording that is very prone to error. As such, we extend the landmark duration without increasing the number of landmarks. For every 20 loops of the program, we start the timer before the first loop and end the timer after the 10^{th} loop. Since the total number of landmarks remains the same as with a sampling rate of 1 in 20, we do not introduce any additional overhead in this process. The Python code is shown below.

```
sampling_rate = 20
interval = r2//sampling_rate
stop_interval = interval//2

...

for k in range(r2):
    # start_instrument
    if i % interval == 0:
        start = timer()
    m3[i][j] += m1[i][k] * m2[k][j]

    # end_instrument
    if i % interval == stop_interval:
        end = timer()
        log.write('{start} {end}\n'.format(start=start, end=end))
```

The timer used is a monotonic clock from the Python `timeit` module. On the Ubuntu server, the timer has a precision to 1 nanosecond.

5.2 Generating the Billing Model

To generate the model, we need the trusted reference node to run the instrumented program with various input sizes. However, it is clear that there is an overhead associated with generating the model. This overhead is a one-time overhead that will ease over the long run as more comparisons are made to available values. In this section, we investigate how a model can be constructed, and how much this overhead would have to be for a sufficiently accurate model. The independent variable is the number of landmarks. The dependent variable is the running time of the process, which directly translates to billing amount via the billing policy.

First, we need to generate a fully-accurate model that serves as the benchmark of accuracy for other models. Due to limitations with equipment and the large running time of an $O(n^3)$ program, we used 200 input cases with matrix sizes randomly ranging from 20 to 80. The code used to build the model can be found in Appendix C. The distribution of randomly-generated input sizes is shown in Figure 5.1. Generally, the distribution is fairly uniform.

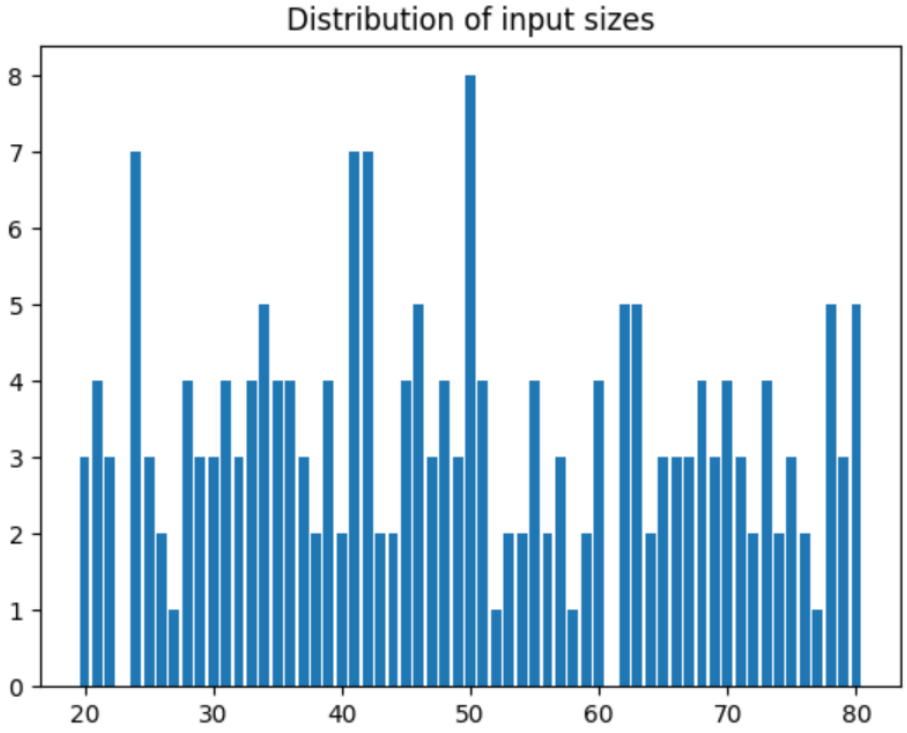


Figure 5.1: Distribution of Input Sizes

From the theoretical analysis, the running time is $O(n^3)$ and the landmark count is of the order $\frac{1}{20} * O(n^3) = O(n^3)$. We expect a linear relationship between the number of landmarks and the running time. This was the graph obtained:

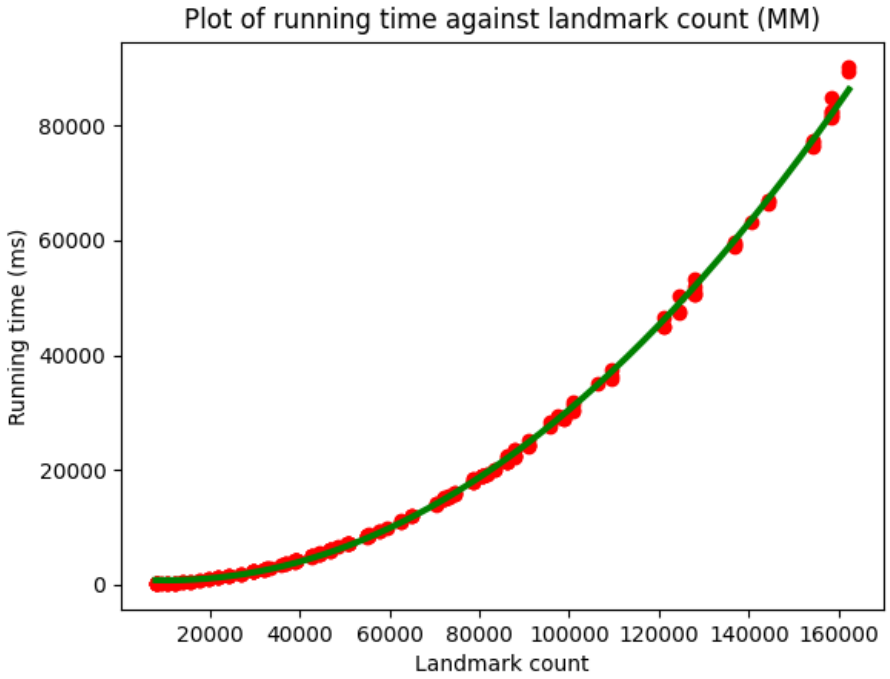


Figure 5.2: Plot of Running Time against Landmark Count

The linear assumption was not suitable for our data. This misfit could be due to the relatively small input size of matrices used. Due to resource constrains, we were not able to increase the load and run on larger input sizes. As such, we switched to use a polynomial model with degree 2. The model is defined by the coefficients shown below.

Term	x^2	x	1
Coefficient	$3.72e-06$	$-7.86e-02$	$1.19e+03$

Table 5.1: Coefficients of Running Time against Landmark Count

In this model, the data points do not vary significantly from the predicted curve. Hence, reducing the number of input instances that have to be pre-run will not significantly affect the results. The only requirement is that the input sizes cover a sufficiently wide range. The

landmark count model is useful for determining **how much work was done**. However, it does not tell **how fast the work was done**.

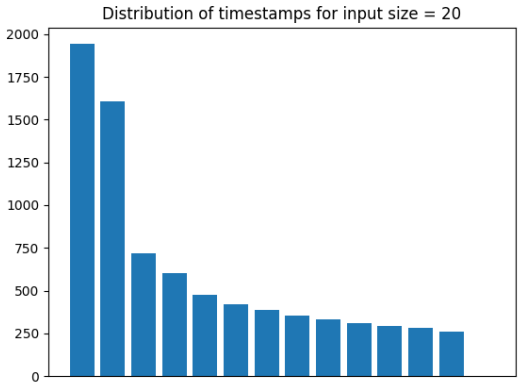
For a verification of the QoS, we need to use the second metric recorded, the total landmark duration. Unfortunately, we are not able to produce a model of this metric. Since the landmark was only encompassing one line of code, its execution time was extremely negligible. On the system clock, many landmarks would record the same start and end time. This behaviour was expected because a landmark is expected to be within a basic block. Often, processes have multiple basic blocks, with each basic block only running for a short period of time. While we are not able to generate a model from the total landmark duration, we are still able to use this metric to detect billing anomalies.

5.3 Estimating the Landmark Running Time

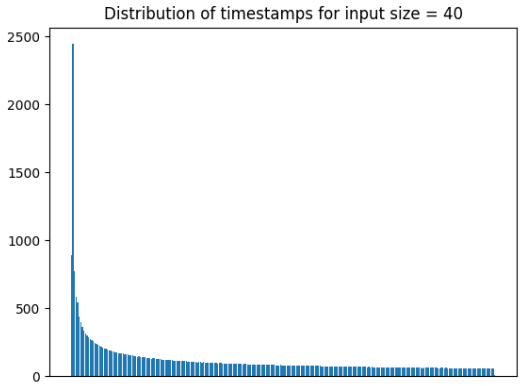
As mentioned in the previous section, our result turned out have the problem of timestamps being too close to each other. However, over tens of thousands of landmarks, the timestamp recorded eventually increased. We can use the number of timestamps in one second to estimate the running time of a landmark. For this task, we choose four particular inputs of sizes 20, 40, 60 and 80. We feed these inputs to our program, and generate the trusted logs. We then count the number of each time stamp and plot the results for the four sizes in Figure 5.3 below.

For all of the inputs, we see that there is an initial spike in the number of landmarks per nanosecond. The difference is due to how the process is scheduled. In the Ubuntu system where we ran these tests, a Completely Fair Scheduler (CFS) is used. The process with the lowest `vruntime`, or the lowest running time on the processor so far, is being scheduled. Overtime, the instrumented process had very low priority on the system resulting in a tapered decrease in the number of landmarks per second.

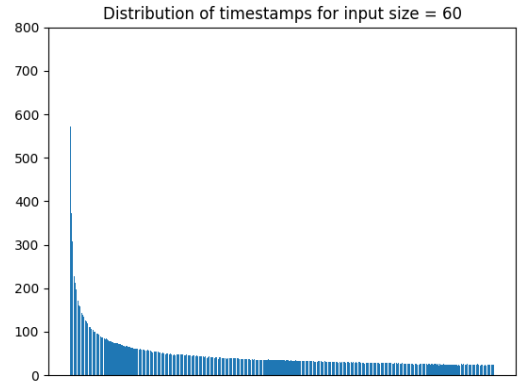
Since the number of landmarks being executed per second is not consistent, then we cannot get an average landmark duration that will be fair for comparison. However, we can still compare the distribution of timestamps for an attacker and from the reference node.



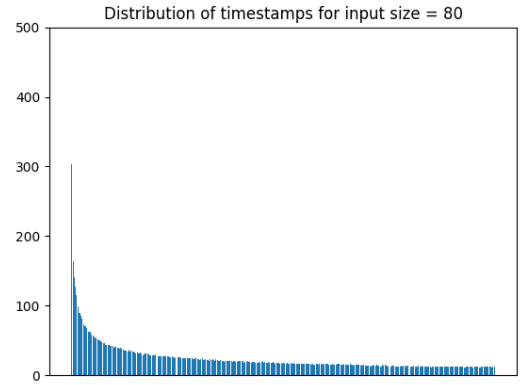
(a) n = 20



(b) n = 40



(c) n = 60



(d) n = 80

Figure 5.3: Distribution of Chronological Timestamps at 90% CPU limit

5.4 Trusted Log with Consistent QoS Degradation

There are many ways that an attacker can degrade the QoS, but the simplest is to have consistent QoS degradation across landmarks and non-landmarks. On the compute node, we use `cpulimit` to degrade the QoS. `cpulimit` works by specifying the maximum CPU usage a process can take up. We found that the slowdowns were easily detectable. With a CPU limit of 90%, the $n = 20$ input instance took a running time of $303ms$, incurring a 50% overhead compared to the reference node's $200ms$. If using a model that was based just on the running time of the process, the client would have been charged 50% more than what was fair. For completeness, we present the additional charges incurred at 90% CPU limit in the table below.

n	**Reference Node Time (ms)**	**Compute Node Time (ms)**	**Overhead (%)**
20	200	303	51.5
40	3691	4017	8.8
60	19508	22777	16.8
80	69011	77739	12.6

Table 5.2: Running Time Overhead with `cpulimit` of 90%

The general pattern is that the extra charges incurred decreases as input size increases. The long-term effect of a CPU-intensive process running is that it will have less priority on the scheduler. As a result, even without `cpulimit`, the $n = 80$ input already incurred some forms of CPU limits from the scheduling.

For such large discrepancies where the billing inflation is significant, the trusted log would very easily reveal the inflated running time. Initially, we thought that multiple landmarks would still run within a timestamp and we would have to compare the two distributions. However, the slowdown resulted in each landmark taking between $160ns$ and $220ns$, a significant slowdown. In all, the trusted log allows the client to dispute a bill with evidence.

Chapter 6

Memory Optimisation

When working on the project, we realised an optimisation which we did not perform. The impact of the unoptimised code caused a very large overhead, which we explain more in Chapter 8. Previously, we were building a **large in-memory log** and only did I/O once. This design decision was from the assumption at the binary level, where we know that I/O functionality is limited for enclaves and hence we want to limit the number of I/O operations.

```
log += 'S1 {start} {end}'.format(start=start, end=end)
```

We realised only later that this would result in a large memory usage to store the log and slow down the process on the CPU. As such, we added the memory optimisation by **freeing the log at each landmark**. Implementing this change significantly reduced the process running time.

```
log.write('S1 {start} {end}'.format(start=start, end=end))
```

6.1 Generating the Billing Model

Having a decreased running time also meant that the processes had a fairer chance at the scheduler, since their `vruntimes` would not be exaggerated. The result was that there was no more clustering of timestamps previously seen. With this improved logging method, we attempt

to generate a model for the running time again. We managed to get a more realistic linear result for the running time against the number of landmarks.

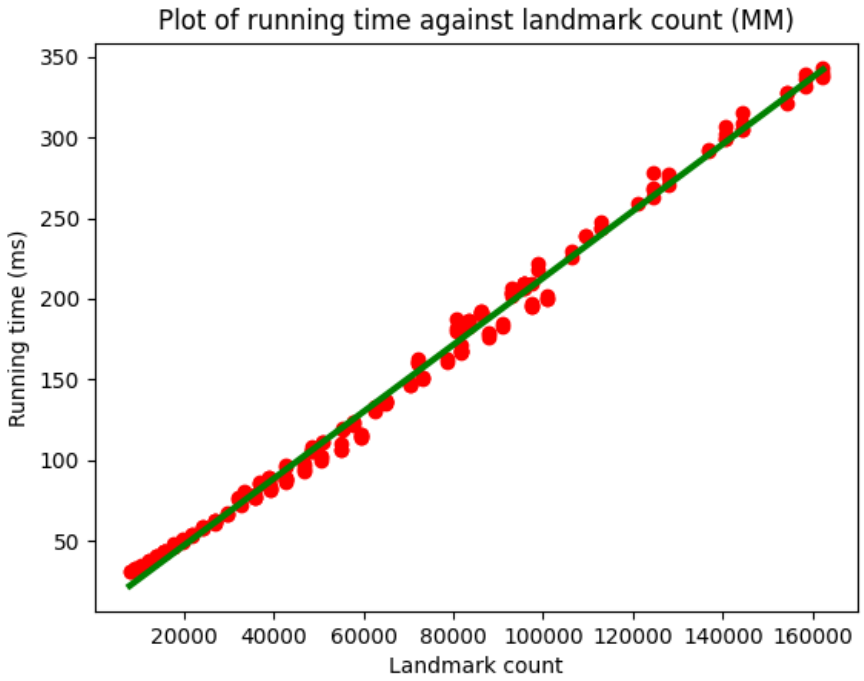


Figure 6.1: Plot of Running Time against Landmark Count with Memory Optimisation

The coefficients of this model are shown below.

Term	x	1
Coefficient	$2.08e-03$	$5.38e+00$

Table 6.1: Coefficients of Running Time against Landmark Count with Memory Optimisation

This linear result here means that the previous quadratic curve we got was a result of the memory overhead. As the number of landmarks and hence number of logs increased, the memory requirement was exponential. We then plot the running time over the total landmark duration (which we were previously not able to do), and get the following result:

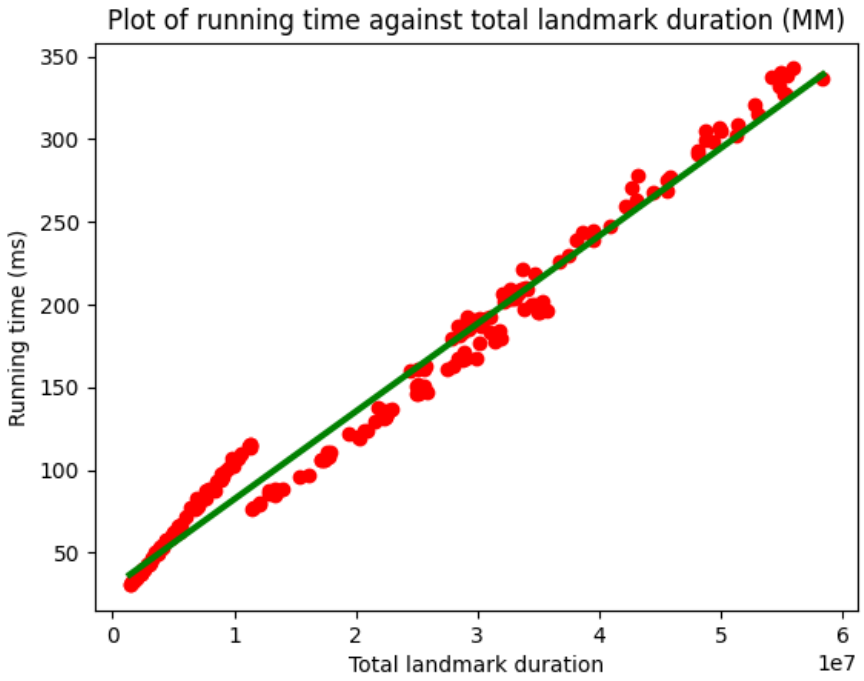


Figure 6.2: Plot of Running Time against Total Landmark Duration with Memory Optimisation

We also get a fairly linear model with the following coefficients:

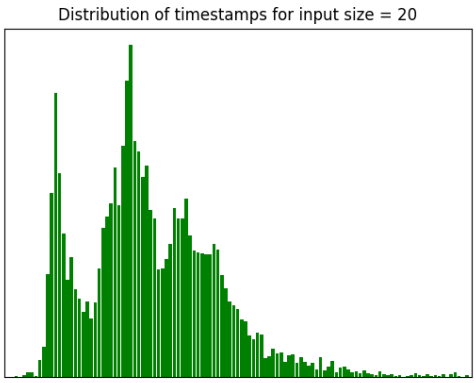
Term	x	1
Coefficient	$5.31e-06$	$2.98e+01$

Table 6.2: Coefficients of Running Time against Total Landmark Duration with Memory Optimisation

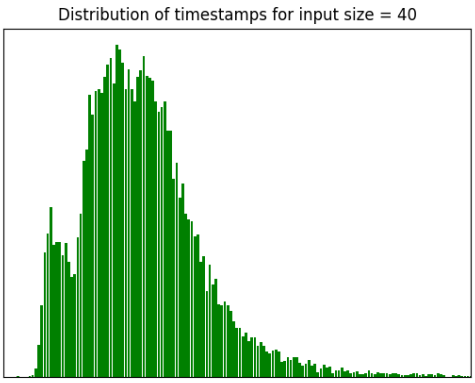
Interestingly, over multiple runs, we consistently observe a dip in the running time as the total landmark duration passes the 1-second mark. The dip could be due to cache optimisations for the code. Before the dip, the cache was not full utilised as it was not filled.

6.2 Estimating the Landmark Running Time

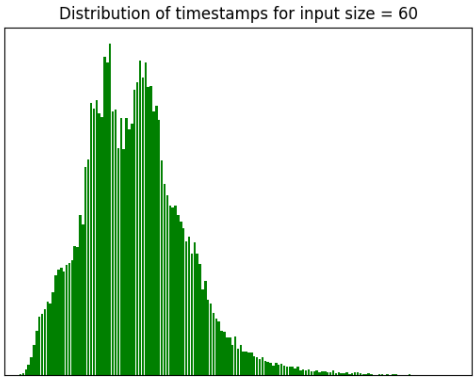
Next, we want to apply the landmark pattern matching method we described in Section 3.4. We first mapped the distribution of landmark times for $n = 20, 40, 60, 80$ using this new imple-



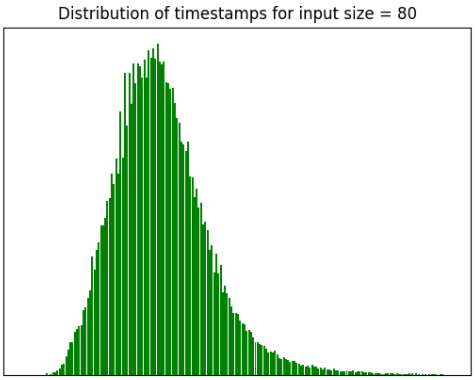
(a) n = 20



(b) n = 40



(c) n = 60



(d) n = 80

Figure 6.3: Distribution of Landmark Duration (Reference Node)

mentation. As shown in Figure 6.3 below, the landmarks times are distributed in a Gaussian manner. Hence, we decide to use the peak value (mode landmark duration) of each process as an indication of the true mean.

Theoretically, one would expect that the landmark duration is constant regardless of the input size, since each landmark is doing the same task of one multiplication. However, in reality, there is process scheduling at play which affects the landmark duration. For each input size that we have, we took the mode landmark duration and then created the following model for estimate average landmark duration over input size. As shown in Figure 6.4, the mode is at 200ms for input sizes $n = 20$ to $n = 39$. From $n = 40$ to $n = 79$, the mode duration is 400ms.

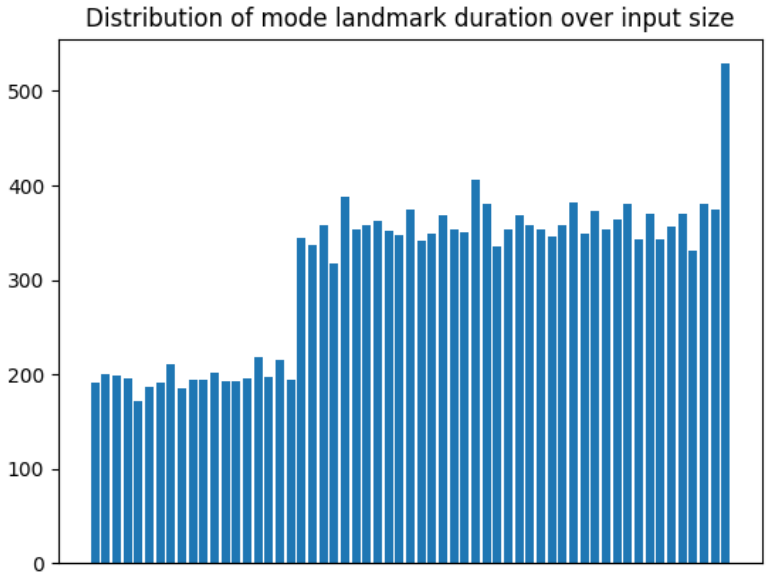
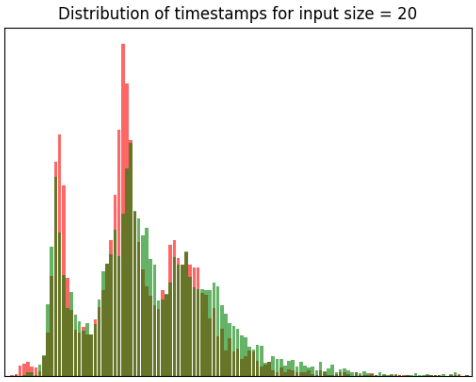


Figure 6.4: Plot of Mode Landmark Duration over Input Sizes

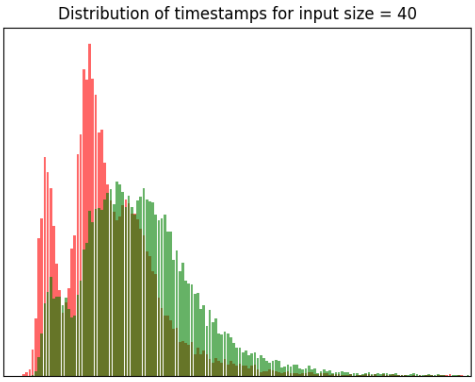
6.3 Trusted Log with Consistent QoS Degradation

The next step is to simulate the attack. As we did in the previous example, we used `cpulimit` to perform consistent QoS degradation at 90% of the CPU limit. We plot the distribution of landmark times for $n = 20, 40, 60, 80$, to compare to what we have in Figure 6.3 above. The red sections correspond to the distribution from the attacker node.

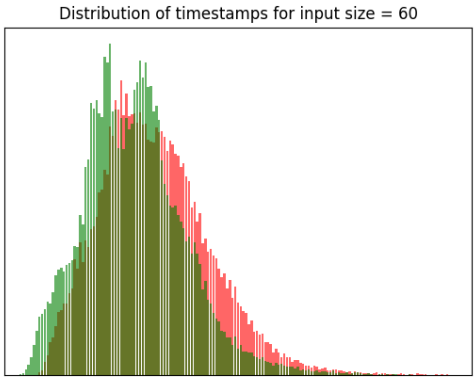
For $n = 80$, we can observe clearly that the attacker's landmarks are more skewed towards landmark with a longer running time. The mode landmark duration has also clearly shifted right. We also see this shift for $n = 60$. We have contradictory results from $n = 20, n = 40$. The attacker node with CPU limit seemed to have a shorter landmark duration. This observation contradicts with how the attacker process had a longer running time of $104.1ms$ than the reference node's $75.7ms$. Over multiple runs and multiple input matrices, we consistently got similar results that there was a preferentially quicker running time during the landmarks. It could be related to how the processes are scheduled in the Ubuntu server, an issue which we have mentioned a few times but did not control for when conducting our experiments.



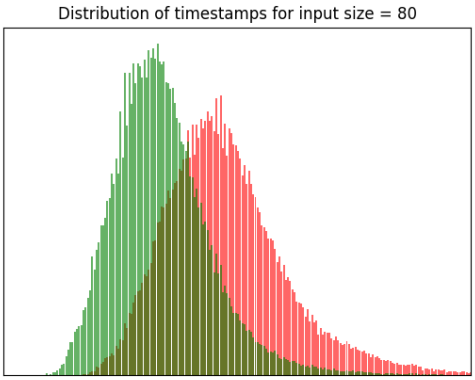
(a) n = 20



(b) n = 40



(c) n = 60



(d) n = 80

Figure 6.5: Distribution of Landmark Duration (90% CPU Limit Node)

6.4 Landmark Pattern Matching

Based on the results earlier, we show the timestamp comparison for a landmark pattern of size 5 for the input size $n = 80$. Generally, we would recommend a larger pattern to match since it is available. However, we use a shorter period for brevity in explanation. We pick the landmark pattern randomly from the middle 40% to 80% of the logs, skipping the initial spike in running speed and the outliers with high running time. In Appendices D and E, we have the full log and the landmark durations extracted from the log respectively. We calculate the following metrics from the information. These metrics are to be used in conjunction with the pre-agreed billing policy to decide how billing disputes should be settled.

6.4.1 Net Adjusted Deviation

The net adjusted deviation is how long more the attacker took for non-landmarked times than the reference node. This value is given by iterating over non-landmark sections in Appendix E and finding: $\Sigma(r_i - q_i)$. Result: $(2012 - 1906) + (1697 - 1680) + (1615 - 1631) + (1620 - 1604) = 123ns$, over a 5-landmark pattern.

6.4.2 Proportion of Time with Inflation

We calculate the various inflation rates in Appendix F. The proportion of time with inflation is the weighted sum over r_i. Result:

$$\frac{1906+543+1680+562+519+1604+502}{526+1906+543+1680+562+1631+519+1604+502} = 77.2\%$$

6.4.3 Inverse Harmonic Mean of Inflation Rates

$$\frac{1}{HM(5.6,10.1,1.0,6.9,13.7,1.0,6.6)} * 100 = 37.8$$

We can also weigh the inflation values by the reference duration of the respective sections.

Chapter 7

Multiple Landmark Types

In this chapter, we detail some concerns with using other types of programs that may not be as straightforward. In particular, we are interested in programs with multiple landmark types. Then, we cannot simply consider the total landmark duration and total number of landmarks against running time as these metrics do not account for different landmark type.

For this analysis, we will use the recursive `FastExponentiation` algorithm. Fast exponentiation is used because of we can identify two distinct branch types in the program. Furthermore, the execution order of the landmarks varies depending on the input. Lastly, while the algorithm is $O(logn)$, the execution time is not directly proportional to the input value n. A large input that is a power of 2 could have a comparable input to a smaller input of the form $2^n - 1$.

7.1 Landmark Selection for Recursive Programs

Recall that our definition for landmark states that a landmark must be within a basic block, so function calls cannot be included within landmarks. Hence, in a recursive fast exponentiation algorithm the following landmark placement is **incorrect**:

```
if e % 2 == 0:
    start = timer()
    # incorrect landmark placement including a function call
    result = fast_exponentiation(x * x, e // 2)
    end = timer()
```

```
    # landmark S1
    log += 'S1 {start} {end}'.format(start=start, end=end)
```

Instead, we extract the landmark by considering all the binary instructions happening just before the function call. The following shows where the instrumentation points are.

```
if e % 2 == 0:
    # Landmark S1
    start = timer()
    x_squared = x * x
    e_halved = e // 2
    end = timer()
    log += 'S1 {start} {end}'.format(start=start, end=end)
    return fast_exponentiation(x_squared, e_halved)
else:
    # e % 2 == 1:
    # Landmark S2
    start = timer()
    e_minus_one = e - 1
    end = timer()
    return n * fast_exponentiation(x, e_minus_one)
```

7.2 Weightage of Multiple Landmarks

Both landmarks shown are different and would result in a different number of binary instructions being executed. When generating the model for running time against the landmark count and total landmark duration, we need to weigh the landmarks. The weight is the number of instructions in the landmark, which is determined statically as we did for the matrix multiplication program in Appendix A.

Given a log, we can calculate the weighed total landmark duration by multiplying each landmark duration with its instruction count. We do the same process for the landmark count as well. The model generated needs to use multi-variable linear regression, where there will be two variables introduced for each landmark type, one for landmark count and one for landmark duration.

Chapter 8

Performance Analysis

The main sources of overhead are from instrumentation, modelling and verification.

8.1 Instrumentation Overhead

We record the average running time of three instances for each input size $n = 20, 40, 60, 80$.

n	**Uninstrumented**	**5% Instrumented**	**Overhead (%)**
20	17	200	1076
40	23	3691	15948
60	39	19508	49921
80	69	69011	99916

Table 8.1: Running Time Overhead due to Instrumentation

As shown in the table, we have an **unacceptably high instrumentation overhead** that only increases with input size. Due to the large volume of I/O performed, we wonder what the overhead due to I/O itself is. There can be some I/O optimisations such as the use of buffers. This is when we realised that problem of building a large in-memory log, and opted for our alternate implementation in Chapter 6. We managed to significantly reduce the running time overheads of the various processes, and the new results are shown below. For each input size, we ran the following input sizes:

- Uninstrumented program
- Instrumented program at 5% of the landmarks
- Instrumented program at all landmarks

n	**Uninstrumented**	**5% Instrumented**	**Full Instrumented**
20	17.1	30.3	31.2
40	23.3	75.7	117.7
60	39.0	161.7	358.8
80	69.8	297.7	922.0

Table 8.2: Running Time (ms) with Memory Optimisation

Firstly we can see that doing 5% of instrumentation only resulted in a significant decrease in overhead compared to doing full instrumentation on all possible landmarks. The savings are all calculated in the table below, with reference to the full instrumented time. As expected, the savings are more significant as input size increases, making savings solution highly scalable.

n	20	40	60	80
Savings (%)	2.9	35.7	54.9	67.7

Table 8.3: Optimisation from using only 5% of landmarks

However, it is of greater interest what the overhead is for instrumenting the programs. The results are in the table below.

n	20	40	60	80
Overhead (%)	77	225	315	327

Table 8.4: Running Time Overhead due to Instrumentation with Memory Optimisation

The overhead is still large and unacceptable. With reference to $n = 60$, a client would have to pay triple the original amount just to have the ability to verify that their process was

executed with a minimum QoS. We believe that more optimisations can be done with regard to I/O, especially at the binary level.

8.2 Modelling and Verification Overhead

With more inputs and larger input sizes, the model can become increasingly accurate. However, the time needed to pre-run all the inputs is heavier. As shown in Figure 5.2, the longest instances of matrix multiplication would take about one and a half minutes to run. Generating the model in 5.2 required about 2 hours on our machine, with most of the time spent on running the inputs. With the memory optimisation, this time is greatly reduced. The instrumentation overhead also contributes to this overhead of having to pre-run all inputs.

Next, as shown in Figure 6.1, the data points did not have a high variance from the predicted linear model. This suggests that having fewer data points will still produce a very similar model, as long as the data points have sufficiently different input sizes to cover the spectrum from $n = 20$ to $n = 80$.

Lastly, the overhead of generating and verifying from the model is very negligible ($< 1\%$).

Chapter 9

Conclusion

9.1 Contributions

In this paper, we present a system for trusted metering by using trusted execution environments. Our system is based on the idea of producing trusted logs with the timestamps of binary-level landmarks in the process. With the number of landmarks in the trusted log, the client is able to verify the amount of work done by the compute node. However, clients need a means to verify the quality of service or processing speed provided. Using the timestamps, we showed how a theoretical case can be made for the usefulness of timestamps.

Then, we showed the work done on binary instrumentation inside the SGX environment, and detail the key issues with current technologies.

Next, we presented an example of the trusted metering system applied to a one-landmark program performing matrix multiplication. We showed how a billing model and verification model can be created. Our system was then placed under various attack scenarios where we simulated a compute node degrading the QoS. With the trusted logs, the client was able to detect a degradation of the QoS provided.

In the midst of working on our system, we also realised a key memory optimisation that can be performed. We repeated our experiments with this optimisation. Following that, we showed the considerations for extending our system design to multiple landmark types, using a fast exponentiation algorithm as an example. In our work, we also showed how the theoretical

system design can be flawed in reality, due to factors like how the processes we run are scheduled.

Lastly, we did a performance analysis, by measuring the overhead of our system. Unfortunately, our system has a very high overhead due to the lack of any optimisations at the binary level or optimisations relating to I/O. Using lower-level techniques like registers will definitely help our case.

9.2 Limitations

Here, we list some key issues regarding the design and implementation of the system.

Static Root of Trust. The design of the entire system trusts the client by making use of a third-party public instrumenting program. We argue that the static root of trust is a necessary reference point for fair billing. Even if we design a system entirely based on trusted hardware, we still need to trust the manufacturer of the hardware, which in our case is Intel. Future designs can consider a scenario where there is no trusted party at all.

Improvement in Binary Techniques. We were very limited in our binary techniques, partly due to lack of support for the SGX instruction set. An entire project can be spent just looking into adding portability to existing binary instrumentation tools. Some of our design choices, such as the use of constant I/O logging, can have high overheads when run inside of enclaves.

Implementation Overhead. With respect to our implementation, a key problem is the high overheads incurred. We need to consider optimisations that can reduce this overhead, especially relating to binary-level optimisations and I/O optimisations.

Attack Simulation. Our attack simulations were all exaggerated. Even at 99% CPU limit, we were achieving new running times running for more than double the original running time. More fine-grained control techniques can be used to experiment if our system design works with small inflation rates. There are also many areas of attack simulation that could be better executed, especially by considering how processes are scheduled in the Ubuntu server. Lastly, we can produce more varied attack scenarios other than consistent QoS degradation.

9.3 Recommendations for Future Work

The key problem with our current implementation is the high overhead. Here, we suggest two directions that a future project may look on.

9.3.1 Use of a Single Timestamp

Our system design involves the use of two timestamps for each landmark. However, we found that the difference in the timestamp could be too small to make meaningful conclusions. A future system can study how a single timestamp can be used per landmark instead. Such a system would need to be evaluated for viability, and new verification metrics have to be designed.

9.3.2 SGX Optimisations

In SGX systems, there is a feature known as switchless OCALLs (Tian, Zhang, Yan, Rudnitsky, Shacham, Yariv, & Milshten, 2018) which we did not investigate in this project. In essence, switchless OCALLs allow enclave processes to make hidden or buffered OCALLs without making a full context switch from the enclave process to the application process. This optimisation can be useful for the I/O heavy task of logging in future, but it is currently a nascent technology.

Besides the use of switchless OCALLs, we can also look to build tools to increase I/O compatibility for SGX systems (Weiser & Werner, 2017). The fundamental principle of SGX security cannot be sacrificed in designing these tools.

Lastly, a known problem with SGX systems is the high overhead of SGX itself. A key contributor to this overhead is the small Enclave Page Cache (EPC) size of only 90 MB, and the long times needed to create and destroy the enclave. This overhead due to SGX can go to 97% (Cai et al., 2020). One area of optimisation is how to make use of the 90 MB cache and reduce page faults.

References

Bruening, D., Zhao, Q., & Amarasinghe, S. (2012). Transparent dynamic instrumentation. *Proceedings of the 8th ACM SIGPLAN/SIGOPS conference on Virtual Execution Environments - VEE 12*, , 2012.

Cai, X., Shi, J., Yuan, R., Liu, C., Zheng, W., Chen, Q., Li, C., Leng, J., & Guo, M. (2020). Outsourcing verification to enable resource sharing in edge environment. *49th International Conference on Parallel Processing - ICPP*, , Aug, 2020.

CRN (2019). Aws billing error overcharges cloud customers. *https://www.crn.com/news/cloud/aws-billing-error-overcharges-cloud-customers*, , 2019.

Dang, H., Tien, D. L., & Chang, E.-C. (2019). Towards a marketplace for secure outsourced computations. *Lecture Notes in Computer Science Computer Security – ESORICS 2019*, , 2019, 790–808.

Duck, G. J., Gao, X., & Roychoudhury, A. (2020). Binary rewriting without control flow recovery. *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, , 2020.

Ince, T., & Hollingsworth, J. K. (2013). Compiler help for binary manipulation tools. *Lecture Notes in Computer Science Euro-Par 2012: Parallel Processing Workshops*, , 2013, 404–413.

Shinde, S., Cui, J., Sen, S., Yuan, P., & Saxena, P. (2020). Binary compatibility for sgx enclaves.

Tian, H., Zhang, Q., Yan, S., Rudnitsky, A., Shacham, L., Yariv, R., & Milshten, N. (2018). Switchless calls made practical in intel sgx. *Proceedings of the 3rd Workshop on System Software for Trusted Execution*, , 2018.

Tople, S., Park, S., Kang, M. S., & Saxena, P. (2018). Verifiable resource accounting using hardware and software isolation. *Applied Cryptography and Network Security Lecture Notes in Computer Science*, , 2018, 657–677.

Tsai, C., Porter, D. E., & Vij, M. (2017). Graphene-sgx: A practical library OS for unmodified applications on SGX. *2017 USENIX Annual Technical Conference (USENIX ATC 17)* (pp. 645–658), Santa Clara, CA, July, 2017: USENIX Association.

Weiser, S., & Werner, M. (2017). Sgxio: Generic trusted i/o path for intel sgx. *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*, , 2017.

Appendix A

Landmark Count

Here, we present the disassembly of the matrix multiplication landmark. Between the landmark indicators, there are 30 instructions. We ignore the overhead caused by the landmark indicator as that value is constant for all landmark types.

```
0x000000000000144a <+266>:   callq  0x1581 <start_instrument()>
0x000000000000144f <+271>:   mov    -0x90(%rbp),%eax
0x0000000000001455 <+277>:   cltq
0x0000000000001457 <+279>:   mov    -0x94(%rbp),%edx
0x000000000000145d <+285>:   movslq %edx,%rdx
0x0000000000001460 <+288>:   add    %rdx,%rdx
0x0000000000001463 <+291>:   add    %rdx,%rax
0x0000000000001466 <+294>:   mov    -0x80(%rbp,%rax,4),%edx
0x000000000000146a <+298>:   mov    -0x8c(%rbp),%eax
0x0000000000001470 <+304>:   cltq
0x0000000000001472 <+306>:   mov    -0x94(%rbp),%ecx
0x0000000000001478 <+312>:   movslq %ecx,%rcx
0x000000000000147b <+315>:   shl    $0x2,%rcx
0x000000000000147f <+319>:   add    %rcx,%rax
0x0000000000001482 <+322>:   mov    -0x40(%rbp,%rax,4),%ecx
0x0000000000001486 <+326>:   mov    -0x90(%rbp),%eax
0x000000000000148c <+332>:   cltq
0x000000000000148e <+334>:   mov    -0x8c(%rbp),%esi
0x0000000000001494 <+340>:   movslq %esi,%rsi
0x0000000000001497 <+343>:   add    %rsi,%rsi
0x000000000000149a <+346>:   add    %rsi,%rax
0x000000000000149d <+349>:   mov    -0x60(%rbp,%rax,4),%eax
0x00000000000014a1 <+353>:   imul   %ecx,%eax
```

```
0x00000000000014a4 <+356>:   add    %eax,%edx
0x00000000000014a6 <+358>:   mov    -0x90(%rbp),%eax
0x00000000000014ac <+364>:   cltq
0x00000000000014ae <+366>:   mov    -0x94(%rbp),%ecx
0x00000000000014b4 <+372>:   movslq %ecx,%rcx
0x00000000000014b7 <+375>:   add    %rcx,%rcx
0x00000000000014ba <+378>:   add    %rcx,%rax
0x00000000000014bd <+381>:   mov    %edx,-0x80(%rbp,%rax,4)
0x00000000000014c1 <+385>:   callq  0x1588 <end_instrument()>
```

Appendix B

Instrumented Program

The full `MatrixMultiplication` program that was instrumented using the method described in Section 5.1.

```
import sys
from timeit import default_timer as timer

fname = sys.argv[1]

log = open(fname + '.trusted-log', 'w')

with open(fname, 'r') as f:
    data = f.read().splitlines()

size1 = data[0].split()
r1 = int(size1[0])
r2 = int(size1[1])
r3 = int(data[r1 + 1].split()[1])

def read_matrix(start, num_rows):
    result = [None] * num_rows
    for i in range(num_rows):
        result[i] = read_row(data[start + i])
    return result

def read_row(row):
    return list(map(int, row.split()))
```

```
m1 = read_matrix(1, r1)
m2 = read_matrix(r1 + 2, r2)

# result has size r1 x r3
def get_row(row):
    result = [None] * r3
    for col in range(r3):
        result[col] = get_cell(row, col)
    return result

sampling_rate = 20
interval = r2//sampling_rate
stop_interval = interval//2

def get_cell(row, col):
    global log
    value = 0
    for i in range(r2):
        # start_instrument
        if i % interval == 0:
            start = timer()

        value += m1[row][i] * m2[i][col]

        # end_instrument
        if i % interval == stop_interval:
            end = timer()
            log.write('{start} {end}\n'.format(start=start, end=end))
    return value

result = [None] * r1
for i in range(r1):
    result[i] = get_row(i)

log.close()
```

Appendix C

Generating the Billing Model

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt

dataframe = pd.read_csv('data.csv', delimiter=' ')
X = dataframe.drop(columns='time')
y = dataframe.iloc[:, -1]

plt.title('Plot of running time against landmark count')
plt.xlabel('Landmark count')
plt.ylabel('Running time (ms)')
plt.scatter(X['num_landmarks'], y, color='red')

coeff = np.polyfit(X['num_landmarks'], y, 2)

from joblib import dump
dump(coeff, 'model.weights')
#coeff = load('model.weights')

ax = np.linspace(min(X['num_landmarks']), max(X['num_landmarks']), 1000000)
y_pred = coeff[0] * ax ** 2 + coeff[1] * ax + coeff[2]
plt.plot(ax, y_pred, color='green', linewidth=3)
plt.show()
```

Appendix D

Landmark Pattern Log

We only consider up to $10^6 ns$ because the timestamps all occurred in the same millisecond. Each entry in the table represents the nanosecond-precision start and end timestamp of a landmark.

Reference Node	**Attacker Node**
(263686, 264212)	(837538, 838057)
(266118, 266661)	(840069, 840667)
(268341, 268903)	(842364, 842965)
(270534, 271053)	(844580 845170)
(272657, 273159)	(846790, 847325)

Table D.1: Logs extracted from the reference node and attacker node

Appendix E

Inter-Landmark Durations

L_i denotes landmark i, and L'_i denotes the non-landmarked time period between L_i and L_{i+1}.

Time Period	**Reference Node** q_i	**Attacker Node** r_i
L_1	526	519
L'_1	1906	2012
L_2	543	598
L'_2	1680	1697
L_3	562	601
L'_3	1631	1615
L_4	519	590
L'_4	1604	1620
L_5	502	535

Table E.1: Landmark durations extracted from the logs in Appendix D

Appendix F

Inter-Landmark Inflation Rates

Time Period	**Inflation Rate s_i**
L_1	-
L_1'	5.6
L_2	10.1
L_2'	1.0
L_3	6.9
L_3'	-
L_4	13.7
L_4'	1.0
L_5	6.6

Table F.1: Inflation extracted from the logs in Appendix D